香港科技大学(广州)
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY (GUANGZHOU)

String Algorithms

- Naive Matching
- Rabin–Karp
- KMP
- Boyer–Moore

Yanlin Zhang & Shangqi Lu | DSAA 2043 Fall 2026

String Matching

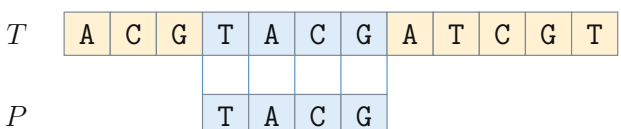
Goal: Find every occurrence of a pattern in a text.

- **Naive matching**: check every alignment.
- **Rabin–Karp**: filter alignments using fingerprints.
- **KMP**: preserve a matched prefix.
- **Boyer–Moore**: use a mismatch and a matched suffix to skip alignments.

What information can we reuse after a comparison?

Applications of String Matching

- Editors and IDEs: search within a document.
- Bioinformatics: find short patterns in long DNA sequences.
- Search and log analysis: search many texts for the same pattern.

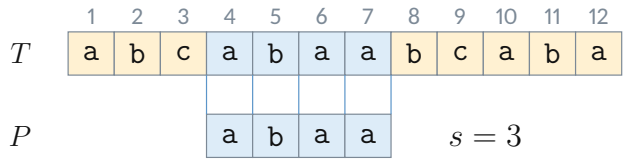


Exact String Matching: Problem Statement

- **Input:** text $T[1..n]$ and pattern $P[1..m]$, with $1 \leq m \leq n$.
- **Output:** all shifts s such that

$$T[s+1..s+m] = P[1..m], \qquad 0 \leq s \leq n-m.$$

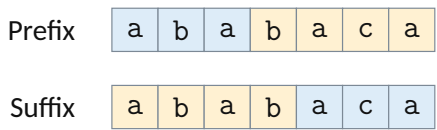
- Shift s means that the occurrence starts at position $s+1$.



The characters must be consecutive. Occurrences may overlap.

Prefix and Suffix

- A **prefix** is a beginning part of a string.
- A **suffix** is an ending part of a string.



- A **proper prefix** is a prefix shorter than the whole string.
- The empty string is a prefix and a suffix of every string.

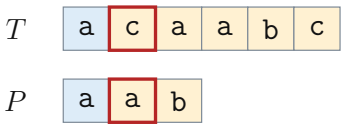
Naive Matcher: Check Every Shift

- Slide the pattern over the text one position at a time.
- At shift s, compare P with $T[s+1..s+m]$ from left to right.
- At each new alignment, restart comparisons at $P[1]$.

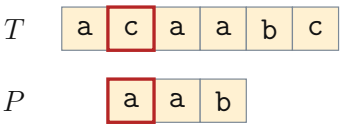
for $s \leftarrow 0$ **to** $n - m$ **do**
    if $P[1..m] = T[s+1..s+m]$ **then**
        report shift s

Naive Matching Example

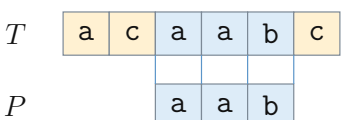
Text acaabc, pattern aab.



$s = 0$: mismatch



$s = 1$: mismatch



$s = 2$: a match

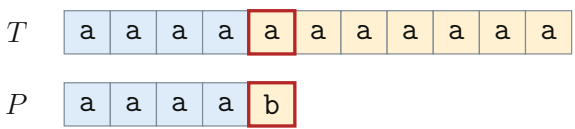
$s = 3$: mismatch

Naive Matching: Running Time

- $n - m + 1$ candidate shifts.
- Up to m character comparisons per shift.

$$\boxed{O\big((n-m+1)m\big)}$$

Worst-case example:



Every shift matches four characters, then fails on the last one.

Rabin–Karp: Fingerprint Idea

Can we reject a text window without comparing all m characters?

- Let f map each length-m string to a short numerical **fingerprint**.
  - Equal strings have equal fingerprints. The converse need not hold.
- At shift s, the text window is $T_s = T[s+1..s+m]$.
- Compare the fingerprints $f(P)$ and $f(T_s)$.

$f(P) \neq f(T_s)$ **Reject** this shift.

$f(P) = f(T_s)$ **Verify** the characters before reporting a match.

A String as a Number

- For an alphabet of d symbols, assign distinct codes $0, 1, \ldots, d-1$.
- Read the string as a number in **base** d.

Decimal digits: alphabet $\{0, 1, \ldots, 9\}$, base $d = 10$.

$$\texttt{31415} \quad \longleftrightarrow \quad 3 \cdot 10^4 + 1 \cdot 10^3 + 4 \cdot 10^2 + 1 \cdot 10 + 5.$$

DNA: $\mathrm{A} = 0$, $\mathrm{C} = 1$, $\mathrm{G} = 2$, $\mathrm{T} = 3$; base $d = 4$.

$$\texttt{CGT} \quad \longleftrightarrow \quad 1 \cdot 4^2 + 2 \cdot 4 + 3 = 27.$$

A Small Fingerprint

The number for a long string may be too large to store directly.

- Keep its remainder modulo a prime q.
- d is the **base**; q is the **modulus**.

$$f(S) = \left(\sum_{r=1}^{m} \text{code}(S[r])\, d^{m-r} \right) \bmod q.$$

Example: pattern $P = 31415$, with $d = 10$ and $q = 13$.

$$f(P) = 31415 \bmod 13 = \mathbf{7}.$$

We store the remainder, not the full number.

True Matches and False Positives

Pattern $P = 31415$, with $d = 10$ and $q = 13$.

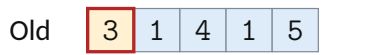
Pattern fingerprint: $f(P) = 31415 \bmod 13 = \mathbf{7}$.

Text window T_s	Fingerprint $f(T_s)$	Decision
14152	8	Different fingerprint: reject
31415	7	Equal strings: report a match
67399	7	False positive: reject

False positive: the fingerprints match, but the strings differ.

Moving the Window by One Position

Computing each fingerprint from scratch would take $O(m)$ time per window.



$$\underbrace{\left(31415 - 3 \cdot 10^4\right)}_{\text{remove the first digit}} \underbrace{\cdot 10}_{\text{shift digits left}} \underbrace{+ 2}_{\text{append the new digit}} = 14152.$$

Reuse the old value instead of recomputing all m terms.

The Rolling Fingerprint

- Precompute $w = d^{m-1} \bmod q$, the first character's weight.
- Reuse $f(T_s)$ to compute $f(T_{s+1})$.

$$\boxed{f(T_{s+1}) = \Big(d(f(T_s) - T[s+1]w) + T[s+m+1]\Big) \bmod q.}$$

In hash arithmetic, each character denotes its integer code.

Example: $T_s = 31415$, $T_{s+1} = 14152$.
With $d = 10$ and $q = 13$: $w = 3$ and $f(T_s) = 7$.

$$f(T_{s+1}) = \Big(10(7 - 3 \cdot 3) + 2\Big) \bmod 13 = 8.$$

A constant number of arithmetic operations per new window.

Rabin–Karp: Initialization

For a length-m string S:

function $f(S)$
  $v \leftarrow 0$
  for $r \leftarrow 1$ **to** m **do**
    $v \leftarrow (dv + S[r]) \bmod q$
  return v

- Compute and store $f(P)$ and $f(T_0)$, where $T_0 = T[1..m]$.
- Precompute $w = d^{m-1} \bmod q$. Initialization takes $O(m)$ time.

Rabin–Karp: Search

Use the stored fingerprints $f(P)$ and $f(T_s)$ at shift s.

for $s \leftarrow 0$ **to** $n - m$ **do**
  if $f(P) = f(T_s)$ **then**
    if $P = T_s$ **then** ▷ Compare the characters
      report shift s
  if $s < n - m$ **then**
    Compute and store
    $f(T_{s+1}) = \Big(d(f(T_s) - T[s+1]w) + T[s+m+1]\Big) \bmod q$

Check the fingerprint, verify a candidate, then roll the window.

Rabin–Karp: Running Time

Let C be the number of shifts with $f(T_s) = f(P)$, including false positives.

- Initialization and rolling updates: $O(m + n)$.
- Character verification: at most $O(mC)$.

$$\boxed{O(m + n + mC)}$$

- Few fingerprint hits mean less character verification.
- Worst case: $O(nm)$, even without false positives if there are many true matches.

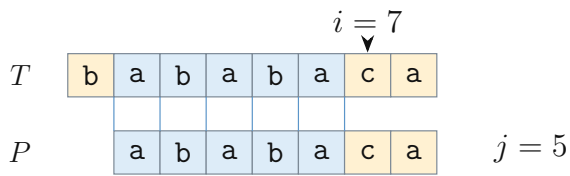
KMP: Recall the Loop Invariant

Knuth–Morris–Pratt (KMP): reuse the prefix already matched.

- i: the **text position** whose character $T[i]$ we compare next.
- j: the length of the matched pattern prefix $P[1..j]$.

$$\boxed{P[1..j] = T[i-j..i-1].}$$

The first j pattern characters match the last j characters of $T[1..i-1]$.

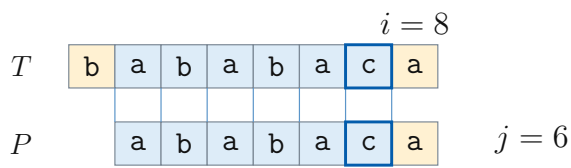


Initialization and a Matching Character

- Initially, $i = 1$ and $j = 0$: the matched prefix is empty.
- If $T[i] = P[j+1]$, extend the match:

$$i \leftarrow i + 1, \qquad j \leftarrow j + 1.$$

Example: $T[7] = P[6] = \texttt{c}$, so $i = 8$ and $j = 6$ after the update.



The same invariant holds for the longer matched prefix.

Mismatch: Preserving the Loop Invariant

Suppose $T[i] \neq P[j+1]$ and $j > 0$.

- We already know $P[1..j] = T[i-j..i-1]$.
- With i fixed, find a smaller j' that preserves the **same invariant**:

$$\boxed{P[1..j'] = T[i-j'..i-1]}, \qquad 0 \leq j' < j.$$

Equivalently, $P[1..j']$ must be a **suffix of the matched prefix** $P[1..j]$.

Choose the largest valid j' to avoid skipping a possible match.

Choosing the Largest Valid j'

- With i fixed, replacing j by j' shifts P right by $j - j'$ positions.
- The **largest valid j'** gives the **smallest shift** that preserves the invariant.

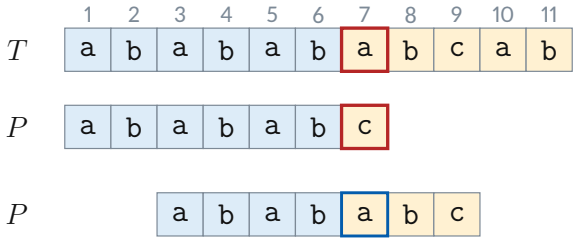
Example: $j = 6$ and the matched prefix is $P[1..6] =$ `ababab`.

Prefix $P[1..j']$	Valid j'	Shift $j - j'$
`abab`	4	$6 - 4 = 2$
`ab`	2	$6 - 2 = 4$
empty	0	$6 - 0 = 6$

Choose $j' = 4$. A smaller j' could skip a possible match.

Example: Choosing $j' = 4$

At $i = 7, j = 6$: $T[7] = \texttt{a} \neq P[7] = \texttt{c}$.

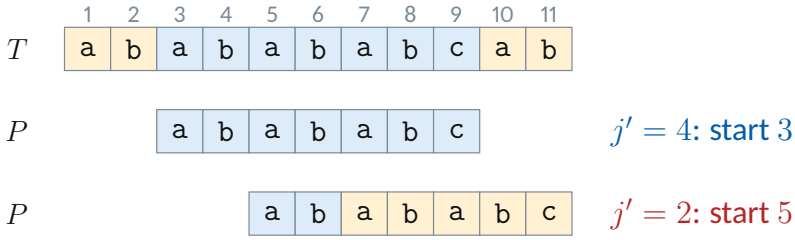


- Largest valid $j' = 4$: shift by $6 - 4 = 2$ positions.
- Same invariant: $P[1..4] = T[3..6] = \texttt{abab}$.

Set $j \leftarrow 4$. Keep $i = 7$ and compare $T[7]$ with $P[5]$: both are a.

Skipping a Match with a Smaller j'

At $i = 7$, $j = 6$, suppose we choose $j' = 2$ instead of 4.



- $j' = 2$ still preserves the invariant: $P[1..2] = T[5..6] = \texttt{ab}$.
- But it skips start 3, where $T[3..9] = P = \texttt{ababab}\texttt{c}$.

Choose the largest valid j' before trying smaller values.

The `next` Function

How can we find the largest valid j' efficiently?

- It depends only on the matched pattern prefix $P[1..j]$.
- Precompute $\mathrm{next}[1..m]$, with $\mathrm{next}[j]$ storing this answer.

$$\boxed{\mathrm{next}[j] = \max\{j' : 0 \le j' < j,\ P[1..j'] = P[j-j'+1..j]\}.}$$

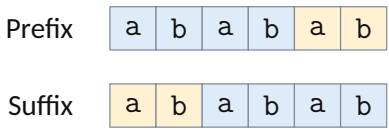
- The length of the **longest proper prefix** of $P[1..j]$ that is also its suffix.
- Value 0 means that only the empty prefix works.

On a mismatch with $j > 0$, set $j \leftarrow \mathrm{next}[j]$ and keep i fixed.
Each update takes $O(1)$ time.

An Example `next` Table

Pattern $P =$ `ababab`c.

r	1	2	3	4	5	6	7
$P[r]$	a	b	a	b	a	b	c
$\text{next}[r]$	0	0	1	2	3	4	0



$P[1..6] =$ `ababab`

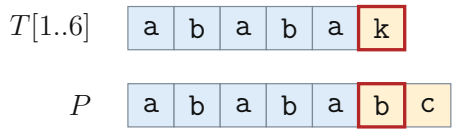
Longest proper prefix that is also a suffix: `abab`

$\text{next}[6] = 4$

For $r = 7$, no nonempty proper prefix ends in c, so $\text{next}[7] = 0$.

Example: Repeated Mismatches

At $i = 6, j = 5$: $T[6] = \texttt{k} \neq P[6] = \texttt{b}$.



Update $j \leftarrow \text{next}[j]$	Same invariant	Retry $T[6] = \texttt{k}$
$5 \to 3$	$P[1..3] = T[3..5] = \texttt{aba}$	$P[4] = \texttt{b} \neq \texttt{k}$
$3 \to 1$	$P[1..1] = T[5..5] = \texttt{a}$	$P[2] = \texttt{b} \neq \texttt{k}$
$1 \to 0$	Both sides are empty	$P[1] = \texttt{a} \neq \texttt{k}$

Choose the largest valid $j' < j$ at each update. Keep $i = 6$ fixed.

The Case $j = 0$

The retained prefix is empty. Compare $T[i]$ with $P[1]$.

- **Match:** set $j = 1$, then advance i by one.
- **Mismatch:** keep $j = 0$, then advance i by one.

In the previous example, $T[i] = \texttt{k} \neq P[1] = \texttt{a}$.

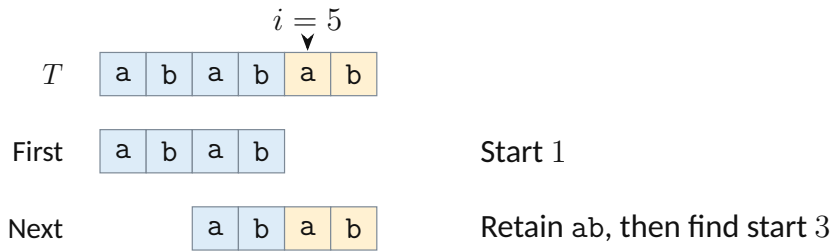
$$i \leftarrow i + 1, \qquad j \leftarrow 0.$$

There is no shorter prefix to try. Move to the next text character.

Complete and Overlapping Matches

When $j = m$, we have found a complete match.
i points to the text position immediately after this match.

- The invariant gives $P[1..m] = T[i - m..i - 1]$.
- Report shift $i - m - 1$ (starting position $i - m$).
- Continue searching: keep i fixed and set $j \leftarrow \mathrm{next}[m]$.



KMP: Search Algorithm

Precompute $\text{next}[1..m]$. At the start of each outer iteration, $0 \leq j < m$.

$i \leftarrow 1; \quad j \leftarrow 0$
while $i \leq n$ **do**
    while $j > 0$ **and** $P[j+1] \neq T[i]$ **do**
        $j \leftarrow \text{next}[j]$
    if $P[j+1] = T[i]$ **then**
        $j \leftarrow j + 1$
    $i \leftarrow i + 1$
    if $j = m$ **then**
        report shift $i - m - 1$
        $j \leftarrow \text{next}[m]$

Running Time of KMP

Assume $\mathrm{next}[1..m]$ has already been computed.

- **Match:** advance i. Charge $O(1)$ work to this advance.
- **Mismatch:** shift P right by at least one position. Charge $O(1)$ work to this shift.

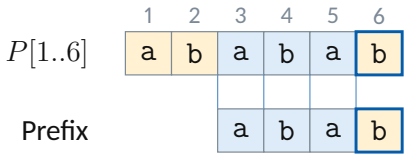
i and the pattern's starting position $i - j$ never move left.

- i advances at most n times.
- The pattern's start advances by at most n positions in total.

$O(n)$ search time after preprocessing.

Computing `next`: A Matching Character

Compute $\mathrm{next}[6]$ from $\mathrm{next}[5] = 3$ for $P =$ `ababab`c.

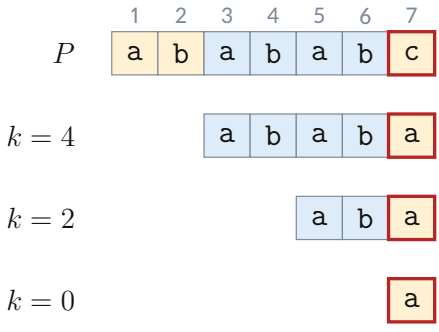


- `aba` is both a prefix and a suffix of $P[1..5]$ ($k = 3$).
- Compare $P[6]$ with $P[4]$. Both are `b`.

$$\mathrm{next}[6] = 3 + 1 = \mathbf{4}.$$

Computing `next`: Different Characters

Compute $\text{next}[7]$ for $P = \texttt{abababc}$; start with $k = \text{next}[6] = 4$.



$P[5] = \texttt{a} \neq \texttt{c}$

$k \leftarrow \text{next}[4] = 2$

$P[3] = \texttt{a} \neq \texttt{c}$

$k \leftarrow \text{next}[2] = 0$

$P[1] = \texttt{a} \neq \texttt{c}$

No extension possible

No candidate can be extended, so $\text{next}[7] = 0$.

Computing `next`: General Rule

Compute $\text{next}[r]$ using the entries already found for $P[1..r-1]$.

- Start with $k = \text{next}[r-1]$.
- The first k characters already match a suffix of $P[1..r-1]$:

$$P[1..k] = P[r-k..r-1].$$

Try to extend this match: compare $P[k+1]$ with the new character $P[r]$.

- Equal: set $\text{next}[r] = k+1$.
- Different and $k > 0$: set $k \leftarrow \text{next}[k]$ and try again.
- Different and $k = 0$: set $\text{next}[r] = 0$.

Computing `next`: Algorithm

Compute the entries from left to right. At iteration r, process $P[r]$.

$\text{next}[1] \leftarrow 0$
for $r \leftarrow 2$ **to** m **do**
  $k \leftarrow \text{next}[r-1]$ ▷ start with the longest matching prefix
  while $k > 0$ **and** $P[k+1] \neq P[r]$ **do**
    $k \leftarrow \text{next}[k]$ ▷ try a shorter prefix
  if $P[k+1] = P[r]$ **then**
    $\text{next}[r] \leftarrow k+1$ ▷ extend the match
  else
    $\text{next}[r] \leftarrow 0$ ▷ $k = 0$ and $P[1] \neq P[r]$

Preprocessing: $O(m)$ time and $O(m)$ space.

Total KMP time: $\boxed{O(n+m)}$.

Practice: A `next` Table

Pattern $P = \texttt{abacaba}$. Complete the second row.

$P[r]$	a	b	a	c	a	b	a
$\mathrm{next}[r]$							

- What is $\mathrm{next}[7]$?
- After a complete match, how far does the pattern move?

Practice: Solution

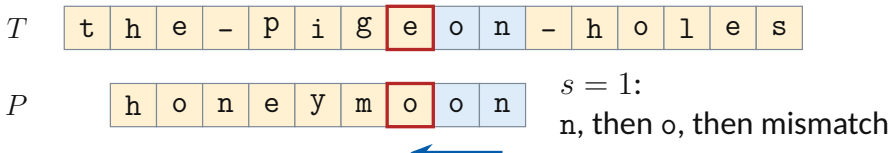
$P[r]$	a	b	a	c	a	b	a
$\text{next}[r]$	0	0	1	0	1	2	3

- $\text{next}[7] = 3$: aba is the longest proper prefix that is also a suffix.
- After a complete match, retain these three characters.
- The next candidate starts $7 - 3 = 4$ positions later.

The retained prefix determines the shift.

Boyer–Moore: Right-to-Left Comparisons

- At shift s, start at **pattern position** $j = m$ and compare $P[j]$ with $T[s + j]$.
- Decrease j after each match. Move P right for the next alignment.

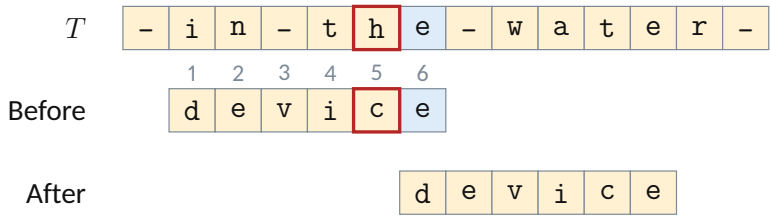


Here _ represents a space in the text.

- **Bad character rule**: use the mismatching text character.
- **Good suffix rule**: reuse an earlier copy of the matched suffix.

Bad Character: Absent from the Pattern

Pattern `device`. The suffix `e` matches, but `c` mismatches `h`.

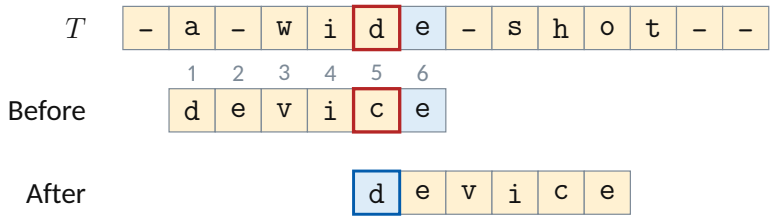


- `h` does not occur anywhere in `device`.
- Move the pattern past this `h`: shift by $j = 5$.

Any smaller shift would still align `h` with a pattern character.

Bad Character: Present in the Pattern

The suffix e matches, but c mismatches the text character d.



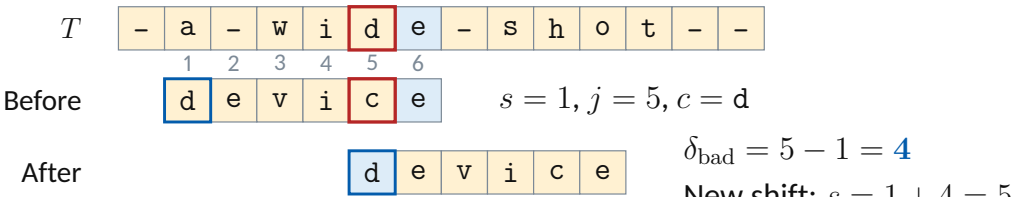
- The rightmost d in the pattern is at position 1.
- Align the pattern's d with the mismatching d in the text. Shift by $5 - 1 = 4$ positions.

The Bad-Character Shift

- At shift s, the **bad character** is $c = T[s + j] \neq P[j]$.
- $\text{last}[c]$: the rightmost position of c in P, or 0 if absent.

δ_{bad}: the number of positions to move P to the right.

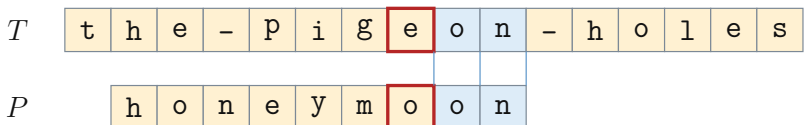
$$\boxed{\delta_{\text{bad}} = \max\{1,\ j - \text{last}[c]\}} \qquad \text{(at least one position).}$$



Good Suffix: Reusing Matched Characters

The **good suffix** is $P[j+1..m]$, already matched before the mismatch at $P[j]$:

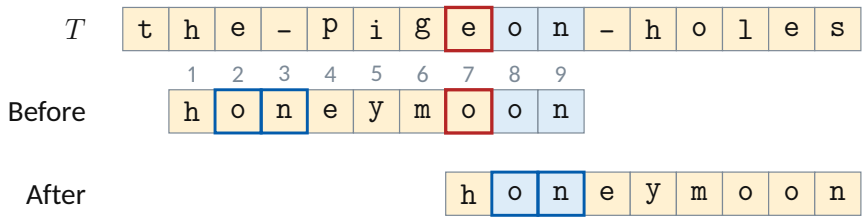
$$\boldsymbol{P[j+1..m] = T[s+j+1..s+m].}$$



- Here $P[8..9] = T[9..10] =$ `on`.
- Any occurrence must agree with this known `on` at shared text positions.

Good Suffix: An Earlier Occurrence

Matched suffix: `on`. Pattern: `honeymoon`.



- The suffix `on` begins at pattern position 8.
- Its rightmost earlier occurrence begins at position 2.

$$\text{Shift} = 8 - 2 = \mathbf{6}.$$

Good Suffix Rule

At a mismatch at $P[j]$ ($1 \le j \le m$), the suffix $S = P[j+1..m]$ has matched.

Precompute $\mathrm{good}[1..m]$ from P:

- $\mathrm{good}[j]$ is the number of positions to shift P right, as suggested by the good-suffix rule for a mismatch at $P[j]$.

If nonempty S has an earlier copy in P, let q be the starting position of its **rightmost earlier copy** ($q < j+1$).

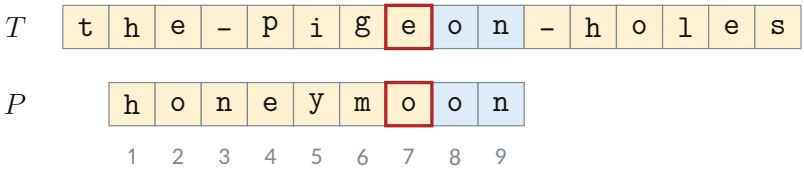
$$\mathrm{good}[j] = \begin{cases} (j+1) - q, & \text{if such a copy exists,} \\ 1, & \text{if } S \text{ is empty or has no earlier copy.} \end{cases}$$

Example: $P = \texttt{honeymoon}$, $j = 7$, $S = \texttt{on}$, $q = 2$:

$$\mathrm{good}[7] = 8 - 2 = \mathbf{6}.$$

Combining the Two Shift Rules

At shift $s = 1$, $P[7] = \texttt{o} \neq T[8] = \texttt{e}$; the matched suffix is on.



- Bad character e: $7 - \text{last}[\texttt{e}] = 7 - 4 = 3$.
- Good suffix on: $8 - 2 = 6$.

$$\delta = \max\{\delta_{\text{bad}},\ \text{good}[7]\} = 6. \qquad s \leftarrow 1 + 6 = 7.$$

Each rule excludes every smaller positive shift, so taking the larger shift is safe.

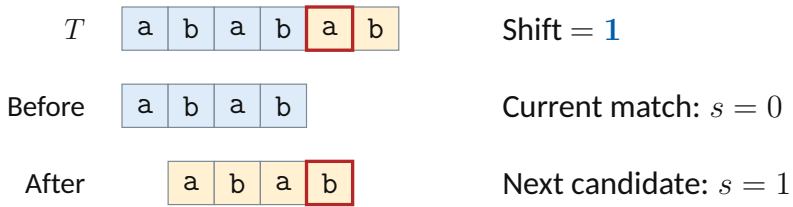
Boyer–Moore After a Complete Match

- If the whole pattern matches ($j = 0$), report shift s.
- Overlapping occurrences are possible. Continue with the next alignment.

$$s \leftarrow s + 1.$$

Move P right by one position; compare again from its right end.

Example: $P =$ `abab` in $T =$ `ababab`.



Boyer–Moore: Precomputed Tables

Precompute once per pattern; use table lookups during the search.
Example: $P = \texttt{honeymoon}$, $m = 9$.

$\text{last}[c]$: the rightmost position of c in P, or 0 if absent.

c	h	o	n	e	y	m
$\text{last}[c]$	1	8	9	4	5	6

$\text{good}[j]$: the good-suffix shift distance for a mismatch at $P[j]$.

j	1	2	3	4	5	6	7	8	9
$\text{good}[j]$	1	1	1	1	1	1	6	6	1

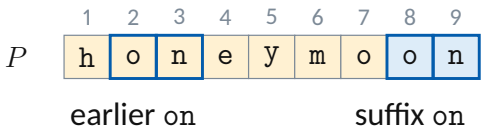
- Mismatch at $j = 7$: suffix $\texttt{on}$ has matched, so $\text{good}[7] = 6$.
- No characters matched ($j = m = 9$): $\text{good}[9] = 1$.

Computing the Boyer–Moore Tables

Build the tables from P before searching the text.

- last: start with zeros. Scan P left to right; set $\text{last}[P[r]] = r$.
- good: start with ones; update entries for suffixes with an earlier copy.

Example: suffix `on` of $P =$ `honeymoon`.



$\text{good}[7] = 8 - 2 = \mathbf{6}$

No earlier copy of `oon`: leave $\text{good}[6] = 1$.

Boyer–Moore: Algorithm

Use the precomputed arrays last and good.

$s \leftarrow 0$
while $s \leq n - m$ **do**
    $j \leftarrow m$
    while $j \geq 1$ **and** $P[j] = T[s + j]$ **do**
        $j \leftarrow j - 1$
    if $j = 0$ **then**
        report shift s
        $s \leftarrow s + 1$ ▷ continue after a complete match
    else
        $c \leftarrow T[s + j]$
        $\delta_{\text{bad}} \leftarrow \max\{1,\ j - \mathrm{last}[c]\}$
        $s \leftarrow s + \max\{\delta_{\text{bad}},\ \mathrm{good}[j]\}$

Boyer–Moore: Running Time

Search phase, with last and good already computed:

- At most $n - m + 1$ alignments.
- At most m character comparisons per alignment.
- **Worst-case search time:** $O(nm)$.

Example: $T = \texttt{aaaaaaaaaa}$, $P = \texttt{aaaa}$.

Every alignment matches all m characters, then shifts by one position.

$$4 \times (10 - 4 + 1) = \mathbf{28} \text{ comparisons.}$$

Information Reused by Each Algorithm

Algorithm	Reusable information
Naive matching	Nothing from the previous alignment
Rabin–Karp	The previous window’s fingerprint
KMP	The pattern prefix already matched
Boyer–Moore	A mismatching character and a matched suffix